Jsp - Servlet

Chapitre 6: Externalisation du code Java

Dr. Ahmad Khoureich Ka

UADB

L3 D2AW / 2012 - 2013

1. Introduction

2. Jsp Expression Language (EL)
   - Accès aux propriétés d'un objet
   - Objets implicites

3. Les JavaBeans
   - Un exemple de javabeans
   - Déclaration d'un javabeans
   - Initialisation de propriètés
   - Lecture d'une propriété

4. JSP Standard Tag Library (JSTL)
   - JSTL: Core
   - JSTL: Function

Introduction

- Un des objectifs de la technologie Jsp est de séparer le traitement de la requête et la logique métier de la présentation (respect du modèle MVC).
- Les jsp sont essentiellement utilisés pour la présentation.
- Il est recommendé d'éviter au maximum l'utilisation d'élements de scripts dans une page jsp.
- L'externalisation du code Java du jsp facilite grandement son écriture et sa maintenance.

Jsp Expression Language (EL)

- Jsp EL permet en quelque sorte de remplacer l'élément de script `<%= expression %>` dans une page jsp.

- Syntaxe: `${expr}`

- `expr` peut être:
  - une expression arithmétique. Exemple: `${2*(4+6)}`
  - une expression logique. Exemple: `${moyenne >= 10}`
  - l'attribut d'un objet quelconque.
    Exemple: `${etudiant.formation}`
  - l'attribut d'un objet implicite. Exemple: `${param.username}`

Jsp EL

- Lors de la compilation du jsp, l'EL est évalué puis converti en `String`.
- EL est naturellement pris en compte dans la page jsp. IL peut être désactivé en mettant l'attribut `isELIgnored` de la directive `<%@page%>` à `true`.
  Exemple: `<%@ page isELIgnored ="true" %>`

Jsp EL: accès aux propriétés d'un objet

- Les variables accessibles avec EL sont les variables de scope, c'est à dire les attributs attachés à la page jsp (`pageContext`), à la requête (`request`), à la session (`session`) ou au contexte de l'application (`application`).
- Le parser EL recherche une variable donnée successivement parmis les attributs attachés à:
  1. la page jsp,
  2. la requête,
  3. la session,
  4. l'application.

Jsp EL: accès aux propriétés d'un objet

Pour accéder à une propriété d'un objet (variable de scope) avec EL les opérateurs `.` ou `[]` peuvent être utilisés.

Exemple:

- Accéder à la propriété **`formation`** de l'objet **`etudiant`**:

```
${etudiant.formation} ou ${etudiant['formation']}
```

- Accéder au 2 ième élément de l'objet **`listAchat`** qui est un tableau ou une **`Collection`**:

```
${listAchat[1]}
```

Jsp EL: Objets implicites

Liste des objets implicites accessibles avec EL

- **`initParam:`** pour accéder à un paramètre d'initialisation de contexte.
- **`param:`** pour accéder à un paramètre de requête monovalué.
- **`paramValues:`** pour accéder à un paramètre de requête multi-valué.
- **`header:`** pour accéder à un en-tête de requête monovalué.
- **`headerValues:`** pour accéder à un en-tête de requête multi-valué.
- **`cookie:`** pour accéder à un cookie.

Jsp EL: Objets implicites

Liste des objets implicites accessibles avec EL (suite)

- **`pageScope:`** pour accéder à un attribut de la page jsp.
- **`requestScope:`** pour accéder à un attribut de requête.
- **`sessionScope:`** pour accéder à un attribut de session.
- **`applicationScope:`** pour accéder à un attribut de l'application.
- **`pageContext:`** représente le contexte de la page jsp. permet d'accéder aux objets **`servletContext`**, **`session`**, **`request`** et **`response`**.

Remarque: Tous les objets implicites sont des instances de **`java.util.Map`** sauf **`pageContext`** qui est une instance de **`javax.servlet.jsp.PageContext`**.

Les JavaBeans

- Un JavaBeans est une classe Java dont la définition respecte un certain nombre de conventions (spécifications de l'API JavaBeans).
- Ces spécifications sont les suivantes:
  - La classe doit implémenter l'interface **`Serializable`**.
  - Le constructeur de la classe ne doit pas avoir de paramètre.
  - Les propriétés de la classe doivent être accessibles à l'aide d'accesseurs publics **`getter`** et **`setter`**.
- Une propriété javabeans peut être en lecture/écriture, lecture seule ou écriture seule.

Les JavaBeans

- Un accesseur **getter** doit avoir la forme:

```
public typePropriete getNomPropriete() { ... }
```

- Un accesseur **setter** doit avoir la forme:

```
public void setNomPropriete(typePropriete p) { ... }
```

- Utilité du javabeans:
  - contenir la logique métier dans une application jsp
  - permettre d'éviter les éléments de scripts `<%! déclaration %>` et `<% scriptlet %>` dans un jsp.

Un exemple de javabeans

```
package appbeans;
import java.io.Serializable;

public class Etudiant implements Serializable{
   private String nom; //propriété en lecture/écriture
   private String[] notes; //propriété en lecture/écriture
   private String formation; //propriété en lecture/écriture
   private float moyenne; //propriété en lecture seule

   public String getNom() {
      return nom;
   }
   public void setNom(String nom) {//accesseur
      this.nom = nom;
   }

   public String[] getNotes() {//accesseur
      return notes;
   }
   public void setNotes(String[] notes) {
      this.notes = notes;
   }
```

Un exemple de javabeans (suite)

```
    public String getFormation() {
        return formation;
    }

    public void setFormation(String formation) {
        this.formation = formation;
    }

    public float getMoyenne() {
        if(notes == null){
            return -1f;
        }
        float total = 0f;
        for (int i = 0; i < notes.length; i++) {
            total += Float.parseFloat(notes[i]);
        }
        moyenne = total / notes.length;
        return moyenne;
    }
}
```

Déclaration d'un javabeans dans un jsp

- Avant d'utiliser un javabeans dans une page jsp, celui-ci doit être déclaré avec l'action jsp **<jsp:useBean>**.
- Syntaxe:

```
<jsp:useBean id="nomObjetJavabeans" class="NomClasseJavabeans"
    scope="page|request|session|application" />
```

  - **id**: spécifie le nom de l'objet javabeans à créer ou à accéder.
  - **class**: spécifie le nom complet de la classe du javabeans.
  - **scope**: spécifie l'objet (page, request, session ou application) auquel le javabeans est attaché.
    C'est attribut est optionnel, sa valeur par défaut est **page**.

Exemple:

```
<jsp:useBean id="etudiantbean" class="appbeans.Etudiant"/>
```

Javabeans: Initialisation de propriètés

- Les propriétés en lecture/écriture et en écriture seule peuvent être initialisées à l'aide de l'action **`<jsp:setProperty>`**
- Syntaxe:

```
<jsp:setProperty name="nomObjetJavabeans" property="nomPropriété"
  [param="nom" | value="valeur"] />
```

  - **`name`**: spécifie le nom de l'objet javabeans.
  - **`property`**: spécifie le nom de la propriété à initialiser ou **`*`** pour initialiser toutes les propriétés du javabeans avec les valeurs des paramètres de requête portant les mêmes noms.
  - **`param`**: spécifie le nom du paramètre de requête dont la valeur est utilisée pour initialiser la propriété. Cet attribut est omis lorsque: **`property="*"`**, la propriété a le même nom que le paramètre de requête ou l'attribut **`value`** est présent.
  - **`value`**: spécifie la valeur du paramètre. Cet attribut est omis lorsque: **`property="*"`**, la propriété a le même nom que le paramètre de requête ou l'attribut **`param`** est présent.

Javabeans: Initialisation de propriètés

Exemples:

- Initialiser la propriété **notes** du javabeans **etudiantbean** avec le paramètre de requête multivaluée **notes**

```
<jsp:setProperty name="etudiantbean" property="notes"/>
```

- Initialiser la propriété **nom** du javabeans **etudiantbean** avec une valeur quelconque

```
<jsp:setProperty name="etudiantbean" property="nom"
    value="ln expo"/>
```

- Initialiser d'un seul coup toutes les propriétés du javabeans **etudiantbean** avec les paramètres de requête

```
<jsp:setProperty name="etudiantbean" property="*"/>
```

Javabeans: Lecture d'une propriété

- Les propriétés en lecture/écriture et en lecture seule peuvent être lues à l'aide de l'action **<jsp:getProperty>**
- Syntaxe:

```
<jsp:getProperty name="nomObjetJavabeans"
    property="nomPropriété" />
```

  - **name**: spécifie le nom de l'objet javabeans.
  - **property**: spécifie le nom de la propriété à lire.

Exemple:

```
Nom étudiant: <jsp:getProperty name="etudiantbean" property="nom"/>
```

Remarque:

Une propriété de javabeans peut être lue à l'aide de jsp EL.

```
Nom étudiant: ${etudiantbean.nom}
```

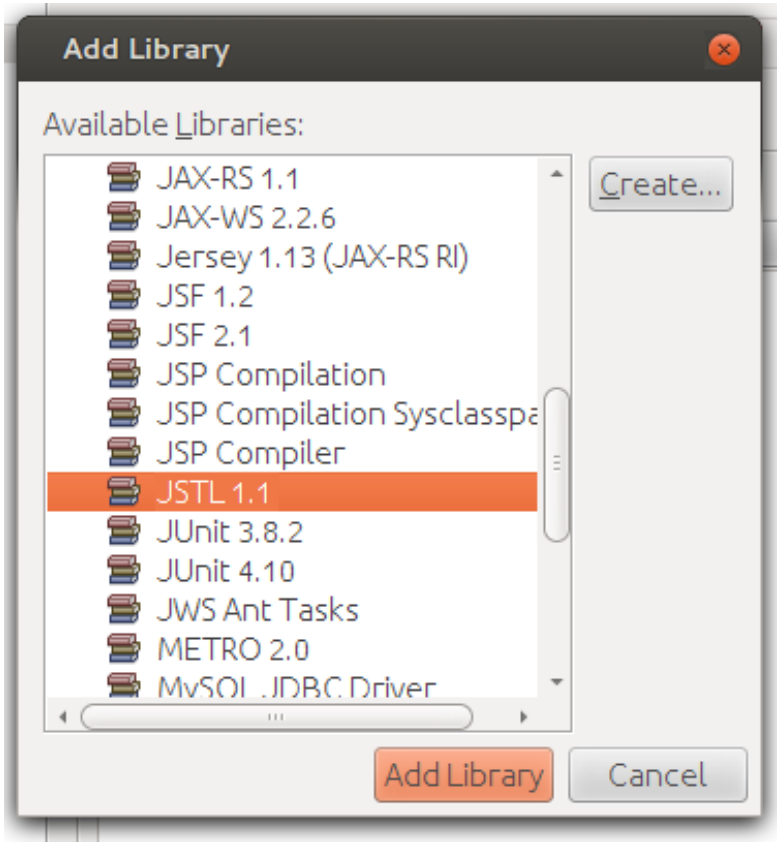
JSTL: JSP Standard Tag Library

- JSTL est un ensemble de librairies de tags (balises) standardisées permettant de remplacer certains traitements qui jusque là n'étaient possibles qu'avec les scriptlets.
- JSTL regroupe 5 librairies de tags qui sont[1] :
  - **`Core:`** librairies de base destinée entre autre à l'utilisation des variables, les structures de contrôle, la gestion des urls.
  - **`Functions:`** pour la lecture de la taille d'une collection et la manipulation de chaine.
  - **`XML:`** pour l'utilisation des documents XML.
  - **`I18n:`** pour l'internationnalisation (langue et convention de formatage spécifiques au client).
  - **`Database:`** pour l'accès aux bases de données.

[1] Dans ce chapitre nous nous limiterons aux librairies Core et Function.

JSTL: JSP Standard Tag Library

- Pour que JSTL soit disponible dans une application web, il suffit d'installer les jars de la JSTL Reference Implementation dans le répertoire **`WEB-INF/lib`** (ou **`libraries`** sous Netbeans).



JSTL: Core

- JSTL Core est la librairie de base. Elle regroupe les tags pour:
  - l'utilisation de variables: **`set`** et **`remove`**.
  - les structures de contrôle: **`if`**, **`choose`**, **`forEach`** et **`forTokens`**.
  - la manipulation des urls: **`url`**, **`redirect`** et **`import`**.
  - d'autres actions: **`out`** et **`catch`**.
- Pour utiliser la librairie JSTL Core dans une page jsp, il suffit de la déclarer avec une directive **`<%@ taglib %>`**

```
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
```

JSTL: `<c:set/>`

- Le tag `<c:set/>` permet de définir une variable dans un des scopes de la page jsp ou de définir la valeur d'une propriété d'un javabeans.
- Syntaxe
  - Définition d'une variable:

```
<c:set var="var" value="valeur"
    [scope="page|request|session|application"] />
```

  - Définition de la valeur d'une propriété:

```
<c:set target="javabeansOuMap" property="nomPropriété"
    value="valeur" />
```

Exemple:

```
<c:set var="pseudo" scope="session" value="${param.pseudo}" />
```

JSTL: `<c:remove/>`

- Le tag `<c:remove/>` permet de supprimer une variable attachée un scope (page, request, session et application) de la page jsp.
- Syntaxe

```
<c:remove var="var" [scope="page|request|session|application"] />
```

Remarque: Si aucun scope n'est précisé pour `<c:remove/>`, la variable à supprimer est recherchée successivement dans `page`, `request`, `session` ou `application`.

Exemple:

```
<c:remove var="pseudo" scope="session"/>
```

JSTL: `<c:if/>`

- Permet d'effectuer un traitement conditionnel comme l'instruction **`if`** en Java.
- Syntaxe

```
<c:if test="condition">
  code jsp ou html
</c:if>
```

- Le corps du tag est exécuté lorsque la valeur de l'attribut **`test`** est **`true`**.

Exemple:

```
<c:if test="${langue == 'fr'}">
  <jsp:forward page="indexFr.jsp"/>
</c:if>
```

JSTL `<c:choose>`

- Le corps du tag `<c:choose>` est constitué d'un ou de plusieurs tags `<c:when>` et optionnellement d'un tag `<c:otherwise>`.
- `<c:choose>` effectue l'exécution du bloc conditionnel constitué du corps du tag `<c:when>` lorsque la condition de celui-ci est évaluée à `true`.
- Le tag `<c:otherwise>` (s'il est présent) est exécuté lorsqu'aucun tag `<c:when>` n'est exécuté.

JSTL <c:choose>

- Syntaxe

```
<c:choose>
  <c:when test="condition1"> code jsp ou html </c:when>
  ...
  <c:when test="condition2"> code jsp ou html </c:when>
  <c:otherwise>
    code jsp ou html
  </c:otherwise>
</c:choose>
```

Exemple:

```
<c:choose>
  <c:when test="${langue == 'fr'}">
    <jsp:forward page="indexFr.jsp"/>
  </c:when>
  <c:otherwise>
    <jsp:forward page="indexEn.jsp"/>
  </c:otherwise>
</c:choose>
```

JSTL `<c:forEach>`

- Le tag **`<c:forEach>`** permet de parcourir une collection d'éléments ou d'effectuer une boucle comme l'instruction **`for`** en Java.
- Syntaxe
  - Faire une boucle avec un compteur allant de **`indiceDébut`** à **`indiceFin`**

```
<c:forEach var="compteur" begin="indiceDébut" end="indiceFin"
    [step="incrément"] [varStatus="étatCompteur"]>
  code jsp ou html
</c:forEach>
```

  - Parcourir une collection d'objets

```
<c:forEach items="collection" var="élément"
    [begin="indiceDébut"] [end="indiceFin"]
    [step="incrément"] [varStatus="étatCompteur"]>
  code jsp ou html
</c:forEach>
```

JSTL `<c:forEach>`

- **`collection`** est une collection d'objet telle que:
  - tableau d'objets ou de primitifs
  - **`java.util.Map`**
  - **`java.util.Collection`**
  - **`java.util.Iterator`**
  - **`java.util.Enumeration`**
- **`varStatus`** est un objet du type **`javax.servlet.jsp.jstl.core.LoopTagStatus`**. Il permet d'avoir l'état du compteur:
  - indiceDébut: **`begin`**
  - indiceFin: **`end`**
  - indiceCourant: **`index`**
  - ...

JSTL <c:forEach>

Exemples:

Afficher les nombres paires entre 0 et 10

```
<c:forEach var="i" begin="0" end="10" step="2">
    <ul>
       <li>${i}
    </ul>
</c:forEach>
```

Parcourir un tableau et afficher ses éléments

```
<c:forEach items="${tableau}" var="elem" varStatus="vStatus">
    <b>Element numéro ${vStatus.index}:</b> ${elem}<br/>
</c:forEach>
```

Afficher les noms et valeurs des paramètres contenus dans la requête. L'objet implicite **param** est un **Map**.

```
<c:forEach items="${param}" var="p" >
    ${p.key} = ${p.value}<br/>
</c:forEach>
```

JSTL <c:forTokens>

- <c:forTokens> permet de parcourir les sous-chaines (séparées par des délimiteurs) d'une chaine de caractères.
- Syntaxe

```
<c:forTokens items="chaineCaractères" delims="delimiteurs"
  var="var" [varStatus="varStatus"] [begin="startIndex"]
  [end="stopIndex"] [step="incrément"]>
  code jsp ou html
</c:forTokens>
```

Exemple:

```
<c:forTokens var="elem" items="20/05/2013-13:34" delims="/-:">
    ${elem}<br/>
</c:forTokens>
```

JSTL `<c:url>`

- `<c:url>` permet (à partir d'une url relative ou absolue) de créer une url avec le correct encodage.
- `<c:url>` inclut l'ID de session à l'url créée si les cookies sont désactivés au niveau du client (réécriture d'url).
- `<c:url>` dispose du sous-tag `<c:param>` pour ajouter des paramètres à l'url créée.

JSTL <c:url>

- Syntaxe

```
<c:url value="url" [context="autreContexteLocal"] [var="var"]
    [scope="page|request|session|application"]>
  <c:param name="nom" value="valeur"/>
</c:url>
```

  - **context** permet de spécifier le contexte d'une autre application déployée dans le même serveur, commence par un slash (/)
  - **var**: variable qui contiendra l'url créée.
  - **scope**: scope auquel **var** est attachée.

Exemple:

```
<c:url value="/discussion.jsp">
    <c:param name="pseudo" value="moussa"/>
</c:url>
```

JSTL <c:redirect>

- Le tag <c:redirect> permet de demander au client de se rediriger vers l'emplacement spécifié par une url.
- Syntaxe

```
<c:redirect url="url" [context="autreContexteLocal"]>
  <c:param name="nom" value="valeur"/>
</c:redirect>
```

- <c:redirect> encode l'url comme le fait <c:url>.
- <c:redirect> peut utiliser le sous-tag <c:param> pour ajouter des paramètres à l'url.

Exemple:

```
<c:redirect url="/ajoutMonnaie.jsp">
    <c:param name="${param.nomMonnaie}" value="${param.valMonnaie}"/>
</c:redirect>
```

JSTL `<c:import>`

- Le tag `<c:import>` permet d'importer la réponse d'une ressource interne ou externe à l'application web.
- La réponse de la ressource peut être insérée dans la réponse courante ou stockée dans une variable attachée à un scope.
- `<c:import>` peut utiliser le sous-tag `<c:param>` pour spécifier des paramètres de requête utiles pour la génération de la réponse de la ressource.

JSTL <c:import>

- Syntaxe

```
<c:import url="url" [context="autreContexte"]
  [var="var"] [scope="page|request|session|application"]
  [charEncoding="charEncoding"]>
  <c:param name="nom" value="valeur"/>
</c:import>
```

Exemple:

```
<c:import url="/dateouHr.jsp" var="date" scope="application">
  <c:param name="affiche" value="date"/>
</c:import>
```

JSTL `<c:out>`

- Le tag `<c:out>` évalue une expression et insère le résultat à la réponse.
- Si le résultat est `null` le tag `<c:out>` peut insérer une valeur par défaut.
- Syntaxe
  - le tag sans le corps

```
<c:out value="expression " [escapeXml="true |false"]
  [default="expression par défaut"] />
```

  - le tag avec le corps

```
<c:out value="expression " [escapeXml="true |false"]>
  expression par défaut
</c:out>
```

  - `escapeXml`: indique si les caractères `<`, `>`, `&`, `'`, `"` doivent être remplacés par leurs codes respectifs : `<`, `>`, `&`, `'`, `"`.

JSTL <c:catch>

- La tag <c:catch> permet d'intercepter les exceptions qui peuvent être lancer pendant l'exécution de son corps.
- Syntaxe

```
<c:catch [var="exception"] >
  code jsp ou html
</c:out>
```

JSTL: Function

- JSTL Function est la librairie pour la lecture de la taille d'une collection et la manipulation de chaine.
- Pour utiliser la librairie JSTL Function dans une page jsp, il suffit de la déclarer avec une directive **<%@ taglib %>**

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/functions"
    prefix="fn" %>
```

- JSTL Function regroupe les tags:
  - **length**: pour la lecture de la taille d'une collection. Cette fonction retourne un **int**.
  - **toUpperCase** et **toLowerCase**: pour modifier la casse d'une chaîne de caractère. Ces fonctions retournent chacune un **String**.

JSTL: Function

- **`substring`**, **`substringBefore`** et **`substringAfter`**: pour extraire une sous-chaine d'une chaine de caractères. Ces fonctions retournent chacune un **`String`**.
- **`replace`**: pour remplacer des caractères dans une chaine de caractères. Cette fonction retourne un **`String`**.
- **`indexOf`**, **`startsWith`**, **`endsWith`**, **`contains`** et **`containsIgnoreCase`**: pour tester si une chaine de caractères contient celle spécifiée. Ces fonctions retournent chacune un **`boolean`** sauf **`indexOf`** qui donne un **`int`**.
- **`escapeXml`**: pour remplacer les caractères **`<`**, **`>`**, **`&`**, **`'`**, **`"`** par leurs codes respectifs : **`<`**, **`>`**, **`&`**, **`'`**, **`"`**. Cette fonction retourne un **`String`**.

JSTL: Function

- **`split`**: pour subdiviser une chaine de caractères suivant un délimiteur. Cette fonction retourne un **`String[]`**.
- **`trim`**: pour supprimer les espaces situés au début et à la fin d'une chaine de caractères. Cette fonction retourne un **`String`**.
- **`join`**: pour créer une chaine de caractères à partir des éléments d'un tableau. La chaine créée contient les élement du tableau séparés par un délimiteur. Cette fonction retourne un **`String`**.

JSTL: Function

Exemples:

Tester l'en-tête **accept-language**

```
<c:choose>
  <c:when test="${fn:startsWith(header['accept-language'], 'fr')}">
    <jsp:forward page="indexFr.jsp"/>
  </c:when>
  <c:otherwise>
    <jsp:forward page="indexEn.jsp"/>
  </c:otherwise>
</c:choose>
```

Remplacer les **/** par des **-**

```
<c:set var="temps" value="${fn:split('12/05/2013', '/')}"/>
<c:out value="${fn:join(temps, '-')}" />
```